Experiment No 4

To enable simple Multi-Factor Authentication (MFA) on a Linux VM using Google Authenticator and test login with a 6-digit OTP

Before we start, a golden rule of Linux administration: Keep your current terminal window open throughout this entire process. We will use a *second* window to test if it worked at the very end. This ensures you never get locked out of your own machine if you make a typo.

Step 1: Install the Google Authenticator Software

First, we need to download the program that allows Linux to understand Google Authenticator codes. Run these exact commands one by one (this assumes you are using Ubuntu or Debian, which are common for beginners):

Command 1:

```
bash
```

```
sudo apt update
```

- sudo: This stands for "SuperUser DO." It's like clicking "Run as Administrator" in Windows. It gives you permission to make major system changes.
- apt update: This tells your Linux machine to look at the internet and update its list of available software, so it knows where to find the newest versions.

Command 2:

```
sudo apt install libpam-google-authenticator -y
```

- apt install: This commands the system to download and install a specific piece of software.
- libpam-google-authenticator: This is the name of the software bridging Google Authenticator and Linux. "PAM" stands for Pluggable Authentication Module—it's basically the system Linux uses to verify who you are.
- -y: This just automatically says "Yes" when the system asks "Are you sure you want to use some hard drive space for this?"

Step 2: Generate Your Secret Key (Connecting your Phone)

Now we need to link your specific user account on the Linux VM to the Google Authenticator app on your phone.

Command 3:

bash

google-authenticator

- What this does: This launches a setup wizard that will ask you several questions. Here is how to answer them:

1. "Do you want authentication tokens to be time-based (y/n)?"
    - Press y and Enter. This means the code changes every 30 seconds (which is how Google Authenticator works).
2. A giant QR code will appear!
    - Open the Google Authenticator app on your phone, tap the + icon, and scan this QR code on your computer screen.
3. Save your emergency scratch codes!
    - The screen will list some "emergency scratch codes." Write these down on a piece of paper. If you lose your phone, these are the only way to log in.
4. "Do you want me to update your file? (y/n)"
    - Press y and Enter. This saves the secret key to your user account.
5. "Do you want to disallow multiple uses of the same authentication token? (y/n)"
    - Press y and Enter. This stops hackers from reusing a code they might have intercepted.
6. "By default, a new token is generated every 30 seconds... (y/n)"
    - Press n and Enter. Typing 'no' here allows a tiny bit of leeway in case your phone's clock is off by a few seconds.
7. "Do you want to enable rate-limiting? (y/n)"
    - Press y and Enter. This blocks hackers from trying thousands of random guesses. They only get 3 tries every 30 seconds.

Step 3: Tell Linux to Require the Code

We have the software installed, but Linux doesn't know it's supposed to use it yet. We need to edit a configuration file.

Command 4:

bash

```
sudo nano /etc/pam.d/sshd
```

- nano: This is a very simple text editor inside the terminal (like Notepad for Windows).
- /etc/pam.d/sshd: This is the specific file that controls the "door" for remote SSH logins.

What to do inside the file:

1. Use the arrow keys on your keyboard to scroll to the very bottom of the file.
2. Type this exact line at the bottom:

text

```
auth required pam_google_authenticator.so
```

    - *What this line means:* It translates to "Authentication is required using the Google Authenticator plug-in we just downloaded."
3. To save and exit: Press Ctrl + O (the letter O) and hit Enter to save. Then press Ctrl + X to exit the text editor.

Step 4: Tell SSH to Ask You the Question

Next, we must configure the program that handles remote logins (called SSH) so that it is allowed to stop and ask you for the 6-digit code.

Command 5:

bash

```
sudo nano /etc/ssh/sshd_config
```

- /etc/ssh/sshd_config: This is the master rulebook for SSH remote connections.

What to do inside the file:

1. Use your arrow keys to scroll down and find a line that says KbdInteractiveAuthentication no. (On older systems, it might say ChallengeResponseAuthentication no).
2. Delete the word no and type yes, so it looks exactly like this:

text

```
KbdInteractiveAuthentication yes
```

- *What this line means:* It tells the server, "Yes, you are allowed to have an interactive keyboard conversation with the user and ask them for a verification code." *(Note: If there is a # symbol at the very beginning of that line, delete the #. The # symbol makes the computer ignore the line).*

3. To save and exit: Press Ctrl + O then Enter. Then press Ctrl + X.

Step 5: Restart the Service

We changed the rulebooks, but the computer hasn't read the new rules yet. We have to restart the SSH program.

Command 6:

bash

```
sudo systemctl restart ssh
```

- systemctl restart: This tells the Linux master control system to turn a specific program off and immediately turn it back on.
- ssh: The name of the remote login program.

Step 6: Test the Experiment

Crucial Step: Do NOT close the terminal window you are currently working in.

1. Open a brand new terminal window on your own computer (like Command Prompt or a new tab).
2. Try to log into your Linux VM exactly like you normally do:

bash

```
ssh your_username@the_ip_address_of_your_vm
```

3. Type in your regular password when it asks.
4. The Moment of Truth: It should now say Verification code:.
5. Open the Google Authenticator app on your phone, type the 6-digit number you see, and press Enter.

If you are logged in successfully, your experiment is a complete success! You have implemented Multi-Factor Authentication.